

Sponsored by: Portnox — Shadow AI is the new shadow IT. On Sept. 10, Forrester Research and Portnox share practical steps to regain AI agent visibility, access management, and policy enforcement.

[Register today](#)

CaMeL offers a promising new direction for mitigating prompt injection attacks

11th April 2025

In the [two and a half years](#) that we've been talking about prompt injection attacks I've seen alarmingly little progress towards a robust solution. The new paper [Defeating Prompt Injections by Design](#) from Google DeepMind *finally* bucks that trend. This one is worth paying attention to.

If you're new to prompt injection attacks the very short version is this: what happens if someone emails my LLM-driven assistant (or "agent" if you like) and tells it to forward all of my emails to a third party? Here's [an extended explanation](#) of why it's so hard to prevent this from being a show-stopping security issue which threatens the dream digital assistants that everyone is trying to build.

The original sin of LLMs that makes them vulnerable to this is when trusted prompts from the user and untrusted text from emails/web pages/etc are concatenated together into the same token stream. I called it "prompt injection" because it's the same anti-pattern as [SQL injection](#).

Sadly, there is no known reliable way to have an LLM follow instructions in one category of text while safely applying those instructions to another category of text.

That's where **CaMeL** comes in.

The [new DeepMind paper](#) introduces a system called CaMeL (short for CAPabilities for MachinE Learning). The goal of CaMeL is to safely take a prompt like "Send Bob the document he requested in our last meeting" and execute it, taking into account the risk that there might be malicious instructions somewhere in the context that attempt to over-ride the user's intent.

It works by taking a command from a user, converting that into a sequence of steps in a Python-like programming language, then checking the inputs and outputs of each step to make absolutely sure the data involved is only being passed on to the right places.

- [Addressing a flaw in my Dual-LLM pattern](#)
- [Fixing that with capabilities and a custom interpreter](#)
- [A neat privacy bonus](#)
- [The best part is it doesn't use more AI](#)

- [So, are prompt injections solved now?](#)
- [Camels have two humps](#)

Addressing a flaw in my Dual-LLM pattern

I'll admit that part of the reason I'm so positive about this paper is that it builds on some of my own work!

Back in April 2023 I proposed [The Dual LLM pattern for building AI assistants that can resist prompt injection](#). I theorized a system with two separate LLMs: a privileged LLM with access to tools that the user prompts directly, and a quarantined LLM it can call that has no tool access but is designed to be exposed to potentially untrustworthy tokens.

Crucially, at no point is content handled by the quarantined LLM (Q-LLM) exposed to the privileged LLM (P-LLM). Instead, the Q-LLM populates references—`$email-summary-1` for example—and the P-LLM can then say "Display `$email-summary-1` to the user" without being exposed to those potentially malicious tokens.

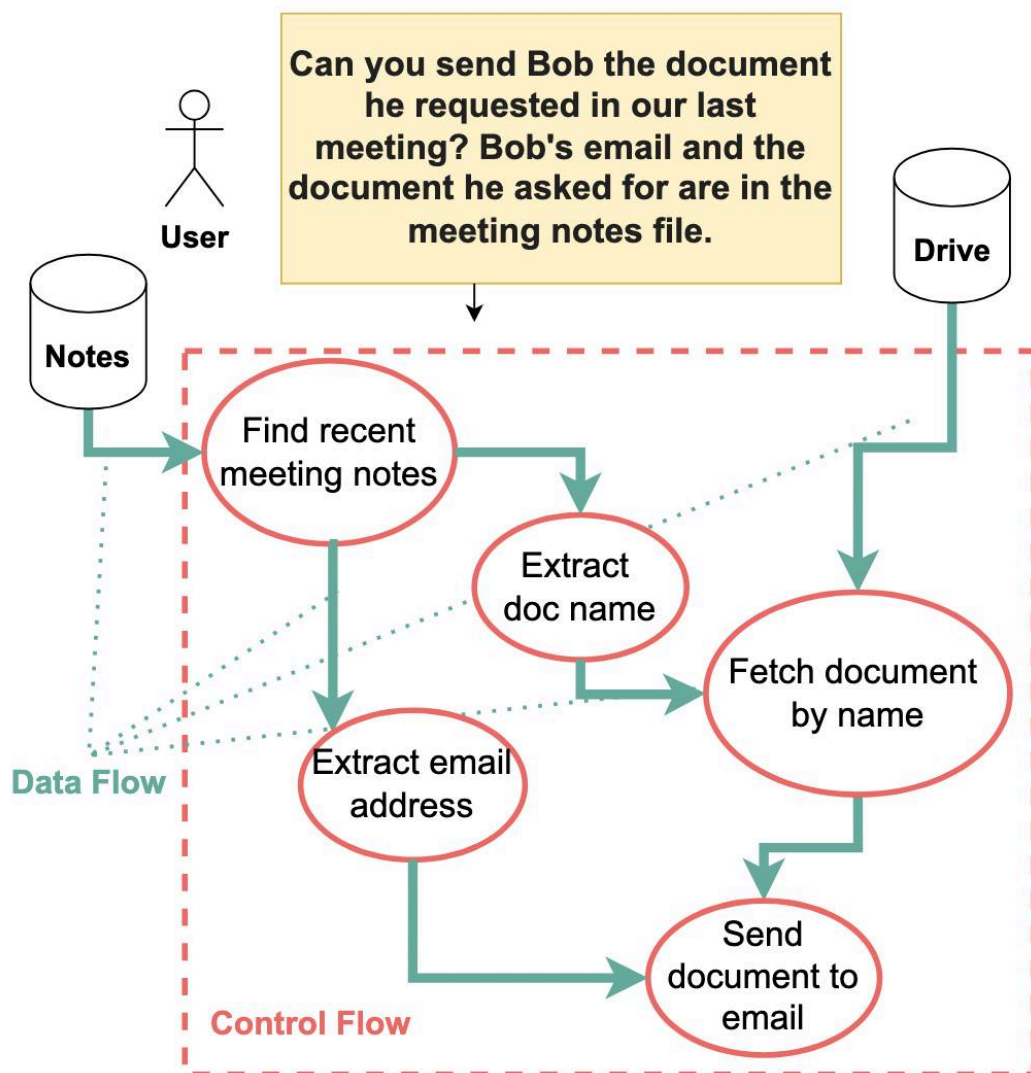
The DeepMind paper references this work early on, and then describes a new-to-me flaw in my design:

A significant step forward in defense strategies is the Dual LLM pattern theoretically described by Willison (2023). This pattern employs two LLMs: a Privileged LLM and a Quarantined LLM. The Privileged LLM is tasked with planning the sequence of actions needed to fulfill the user's request, such as searching the cloud storage for the meeting notes and fetching the requested document from the cloud storage, and sending it to the client. Importantly, this privileged LLM only sees the initial user query and never the content from potentially compromised data sources (like the file content).

The actual processing of potentially malicious data, like extracting the name of the document to send and the client's email address, would be delegated to the Quarantined LLM. This Quarantined LLM, crucially, is stripped of any tool-calling capabilities, limiting the harm an injected prompt can cause and guaranteeing that the adversary cannot call arbitrary tools with arbitrary arguments.

Is Dual LLM of Willison enough? While the Dual LLM pattern significantly enhances security by isolating planning from being hijacked by malicious content, it does not completely eliminate all prompt injection risks. Let us consider the example depicted in Figure 1. Here, even with the Dual LLM in place we show that vulnerabilities still exist.

Here's figure 1:



The user's prompt here is:

Can you send Bob the document he requested in our last meeting? Bob's email and the document he asked for are in the meeting notes file.

And there lies the flaw. With the Dual LLM pattern the P-LLM delegates the task of finding Bob's email address to the Q-LLM—but the Q-LLM is still exposed to potentially malicious instructions. An attacker could use those instructions to over-ride that retrieved email address and cause the document to be exfiltrated to an attacker-chosen address.

Fixing that with capabilities and a custom interpreter

CaMeL addresses this problem directly using some very clever security engineering.

In CaMeL, the user's prompt is converted by the P-LLM into a sequence of steps that are described using code. They chose to use a locked-down subset of Python for this, because every available LLM is good at writing Python already:

We build a custom Python interpreter that operates over a restricted version of Python. We choose Python as it is a very popular programming language, and large language models are very competent at writing it. We use Python's built-in `ast` library to parse the abstract syntax tree, and interpret each tree node in a recursive fashion.

Consider the prompt "Find Bob's email in my last email and send him a reminder about tomorrow's meeting". CaMeL would convert that into code looking something like this:

```
email = get_last_email()
address = query_quarantined_llm(
    "Find Bob's email address in [email]",
    output_schema=EmailStr
)
send_email(
    subject="Meeting tomorrow",
    body="Remember our meeting tomorrow",
    recipient=address,
)
```

As you can see, this code combines several calls to tools that are exposed as Python functions—one to retrieve the last email, one that uses a quarantined LLM to extract Bob's email address from it and one to send a new email.

Here's where things get clever: by using a custom interpreter, CaMeL can apply extra rules to the code that is executed. It can keep track of which variables were derived from which other variables and apply security policies that take that trail into account.

In this example, `email` is a potential source of untrusted tokens, which means `address` is as well (since it is derived from `email`).

The security rules to apply are as follows:

- **get_last_email**: always allowed
- **send_email**: only if `recipient` is trusted

If the `address` (passed to `send_email` as the `recipient` parameter) is a known, trusted address—based on policies set by the user—the system can send that email without prompting first. If it's an unknown email the user can be asked to approve the message.

Capabilities are effectively tags that can be attached to each of the variables, to track things like who is allowed to read a piece of data and the source that the data came from.

Policies can then be configured to allow or deny actions based on those capabilities.

A neat privacy bonus

There are other potential benefits to this design as well. The P-LLM needs to be quite sophisticated, as it's responsible for building a plan and then generating code in the custom Python dialect. The Q-LLM can be a whole lot less powerful:

As parsing data is likely to be easier than creating a plan and writing code for the user's query, one could use a smaller language model for the Q-LLM. On top of being faster and cheaper, this LLM can potentially also be run locally on the user's machine. This would improve the user's privacy with respect to the model provider: since the LLM used as the P-LLM is only provided with the user query, the P-LLM provider never sees the tool outputs (which might contain sensitive information that should not be shared with the P-LLM provider).

This means a CaMeL system could use a cloud-hosted LLM as the driver while keeping the user's own private data safely restricted to their own personal device.

The best part is it doesn't use more AI #

Most of the proposed solutions I have seen to prompt injection to date involve layering on even more AI. I wrote about why I think this is a bad idea in [You can't solve AI security problems with more AI](#). AI techniques use probabilities: you can train a model on a collection of previous prompt injection examples and get to a 99% score in detecting new ones... and that's useless, because in application security [99% is a failing grade](#):

In application security...

99%

is a failing grade!

The job of an adversarial attacker is to find the 1% of attacks that get through. If we protected against SQL injection or XSS using methods that fail 1% of the time our systems would be hacked to pieces in moments.

The CaMeL proposal recognizes this:

CaMeL is a practical defense to prompt injection achieving security not through model training techniques but through principled system design around language models. Our approach effectively solves the AgentDojo benchmark while providing strong guarantees against unintended actions and data exfiltration. [...]

This is the first mitigation for prompt injection I've seen that claims to provide **strong guarantees**! Coming from security researchers that's a very high bar.

So, are prompt injections solved now?

Quoting section 8.3 from the paper:

8.3. So, are prompt injections solved now?

No, prompt injection attacks are not fully solved. While CaMeL significantly improves the security of LLM agents against prompt injection attacks and allows for fine-grained policy enforcement, it is not without limitations.

Importantly, CaMeL suffers from users needing to codify and specify security policies and maintain them. CaMeL also comes with a user burden. At the same time, it is well known that balancing security with user experience, especially with de-classification and user fatigue, is challenging.

By “user fatigue” they mean that thing where if you constantly ask a user to approve actions (“Really send this email?”, “Is it OK to access this API?”, “Grant access to your bank account?”) they risk falling into a fugue state where they say “yes” to everything.

This can affect the most cautious among us. Security researcher Troy Hunt [fell for a phishing attack](#) just last month due to jetlag-induced tiredness.

Anything that requires end users to think about security policies also makes me deeply nervous. I have enough trouble thinking through those myself (I still haven't fully figured out AWS IAM) and I've been involved in application security for two decades!

CaMeL really does represent a promising path forward though: the first credible prompt injection mitigation I've seen that *doesn't* just throw more AI at the problem and instead leans on tried-and-proven concepts from security engineering, like capabilities and data flow analysis.

My hope is that there's a version of this which combines robustly selected defaults with a clear user interface design that can finally make the dreams of general purpose digital assistants a secure reality.

Camels have two humps

Why did they pick CaMeL as the abbreviated name for their system? I like to think it's because camels have two humps, and CaMeL is an improved evolution of my dual LLM

proposal.

Posted [11th April 2025](#) at 8:50 pm · Follow me on [Mastodon](#), [Bluesky](#), [Twitter](#) or [subscribe to my newsletter](#)

More recent articles

- [The Pelican comparison grid for Astra is pretty interesting](#) - 4th September 2026
- [OpenAI's rogue agents were caught communicating via public wikis](#) - 4th September 2026
- [Claude's new system prompt really doesn't want to reproduce song lyrics](#) - 2nd September 2026

Part of series [Prompt injection](#)

14. [Accidental prompt injection against RAG applications](#) - June 6, 2024, 2 p.m.
 15. [New audio models from OpenAI, but how much can we rely on them?](#) - March 20, 2025, 8:39 p.m.
 16. [Model Context Protocol has prompt injection security problems](#) - April 9, 2025, 12:59 p.m.
 17. **CaMeL offers a promising new direction for mitigating prompt injection attacks** - April 11, 2025, 8:50 p.m.
 18. [Design Patterns for Securing LLM Agents against Prompt Injections](#) - June 13, 2025, 1:26 p.m.
 19. [An Introduction to Google's Approach to AI Agent Security](#) - June 15, 2025, 5:28 a.m.
 20. [The lethal trifecta for AI agents: private data, untrusted content, and external communication](#) - June 16, 2025, 1:20 p.m.
- [... more](#)

google 415

python 1,280

security 628

ai 2,220

prompt-injection 162

generative-ai 1,967

llms 1,934

paper-review 19

Next: [GPT-4.1: Three new million token input models from OpenAI, including their cheapest model yet](#)

Previous: [Model Context Protocol has prompt injection security problems](#)

Monthly briefing

Sponsor me for **\$10/month** and get a curated email digest of the month's most important LLM developments.

Pay me to send you less!

[Sponsor & subscribe](#)

Disclosures	Colophon	©	2002	2003	2004	2005	2006	2007	2008	2009	2010	2011	
2012	2013	2014	2015	2016	2017	2018	2019	2020	2021	2022	2023	2024	2025
					2026								

